

S3 Demo — Run Steps

Standing the demo up from a clean checkout, in any sandbox

Generated from demo/DEMO_STEPS.md · repo ams-s3-demo
Companion docs: apps/README.md (what each application is) · demo/DEMO_TEST_GUIDE.md (per-scenario rehearsal script)

How to stand this demo up from a clean checkout, in any sandbox. Written for someone who has never run it before — if you already know the repo, you want demo/DEMO_TEST_GUIDE.md instead, which is the per-scenario rehearsal script.

1. What you are standing up

Four applications, each started by its own script under apps/ , plus the S3 tooling that drives them. You do **not** need all four for every beat.

#	APPLICATION	START	PORT	NEEDED FOR
1	Console — FastAPI + React. The screen you present from.	apps/run-console.sh	8000 + 5173	Every beat
2	PolicyCore — the client's policy portal (Streamlit). The window the audience watches change.	apps/run-policycore.sh	8501	CR-2026-041, CR-2026-042
3	Policy-Service — ClaimsPortal policy side (Spring Boot).	apps/run-policy-service.sh	8081	CR-2026-043
4	Claims-Service — ClaimsPortal claims side (Spring Boot). Start after #3.	apps/run-claims-service.sh	8082	CR-2026-043

OPEN THIS ONE
The console UI is http://localhost:5173 , not :8000 . Port 8000 is the API the UI talks to.

See apps/README.md for what each folder is and how it maps to a ServiceNow application.

2. Prerequisites

TOOL	VERSION	NEEDED FOR	CHECK
Python	3.12+	Console API, PolicyCore, S3 tooling	python3 --version
Node.js	18+	Console UI	node --version
Java JDK	17+	Apps 3 and 4 only	java -version
Maven	3.9+	Apps 3 and 4 only	mvn -version

Skip Java and Maven entirely if you are only demoing the PolicyCore CRs.

3. First-time setup from base packages

From the repository root. Run once per machine.

```
# 1. Python environment
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# 2. Console UI dependencies
cd apps/console/web && npm install && cd ../../..

# 3. Configuration
cp .env.example .env
#   then edit .env – see section 4

# 4. Seed the PolicyCore database
python -m apps.policycore.core.seed

# 5. Confirm the install
python -m pytest -q          # expect: 309 passed
```

SANITY CHECK

If `pytest` passes, the wiring is correct. It exercises the routing registry, the code-review flow and both replay targets without needing an LLM or any of the four applications running.

4. Pointing it at your LLM

Edit `.env`. Pick *one* provider block.

Self-hosted / custom model (most sandboxes)

For any endpoint that speaks the OpenAI chat API — an internal gateway, vLLM, LiteLLM, TGI, LM Studio:

```
LLM_PROVIDER=custom
CUSTOM_LLM_BASE_URL=https://llm.internal.example/v1
CUSTOM_LLM_MODEL=llama-3.3-70b
CUSTOM_LLM_API_KEY=          # optional – blank is fine if your gateway does no auth
```

THE BASE URL IS USED EXACTLY AS WRITTEN

Include the version segment your gateway expects, usually `/v1`. Nothing is appended or rewritten, so a wrong URL gives a clean 404 rather than a confusing one. The client requests

```
<CUSTOM_LLM_BASE_URL>/chat/completions.
```

Verify the endpoint is reachable before demo day:

```
python -c "
from common.llm import complete
print(complete('Reply with the single word: ready'))
"
```

Other providers

PROVIDER	LLM_PROVIDER	ALSO SET
Anthropic	anthropic	ANTHROPIC_API_KEY, ANTHROPIC_MODEL
OpenAI	openai	OPENAI_API_KEY, OPENAI_MODEL
Claude via Bedrock	bedrock	AWS_REGION, BEDROCK_MODEL
Ollama (local)	ollama	OLLAMA_BASE_URL, OLLAMA_MODEL

Replay mode — why the demo survives a bad network

The default is `LLM_MODE=replay`. The code-generation and test-generation beats are served entirely from committed recordings in `s3_enhancement/cache/` and never call your model at all. That is what makes the demo deterministic.

SHARP EDGE — READ THIS ONE.

The *narrative* beats (impact analysis, effort estimate, release notes, design doc) go through a different cache that does **not** honour `LLM_MODE`. On a cache miss they call your endpoint live, even in replay mode. Always run `demo/warm_s3_cache.sh` before presenting — otherwise a slow or unreachable model stalls a beat mid-demo.

5. Reset before every rehearsal or demo

The demo mutates real source files and the SQLite database. Reset returns everything to the pre-CR baseline. Run all three **in this order** — the endorsement reset must come after `reset_s3.sh`, because that script reseeds the database the endorsement baseline then builds on.

```
demo/reset_s3.sh           # CR-2026-041 (PolicyCore coverage tier) + shared state
demo/reset_s3_endorsement.sh # CR-2026-042 (PolicyCore endorsement priority)
demo/reset_s3_springdemo.sh # CR-2026-043 (ClaimsPortal deductible)
demo/warm_s3_cache.sh      # ALWAYS last — reset_s3.sh wipes .cache/llm
```

Expected final lines:

```
S3 source baseline restored, mockapp reseeded, LLM cache cleared, and ticket timeline cleared.
CR-2026-042 source baseline restored, mockapp reseeded, and LLM cache cleared.
Spring demo (ClaimsPortal) source baseline restored; generated files and build output removed.
```

6. Start the applications

One terminal per application. For the PolicyCore CRs you need only the first two.

```
apps/run-console.sh      # terminal 1 — API :8000 + UI :5173
apps/run-policycore.sh   # terminal 2 — portal :8501

# only for the ClaimsPortal CR (CR-2026-043):
apps/run-policy-service.sh # terminal 3 — :8081 (start before claims)
apps/run-claims-service.sh # terminal 4 — :8082
```

Then open `http://localhost:5173` and log in. Any roster name works; passcodes are in `common/roster.py`.

LOGIN	PASSCODE	ROLE
Ravi Kumar	1001	Engineer — App Support, PolicyCore
Priya Nair	1003	Engineer — App Support, ClaimsPortal
Manager	9000	Manager view

7. Optional — seed the routing beat

With the console API already running, this creates a ticket that carries a ServiceNow Configuration Item, so the console can route it to an owning team before any AI step runs.

```
# Default: routes to BillingGateway — an application with an owning team and
# NO repo here. Routing succeeds; automation correctly stays off.
demo/seed_problem_record_ticket.sh

# The other half: routes to a team AND offers the CR to run against it.
SEED_CI=ClaimsPortal SEED_BUSINESS_SERVICE="Claims Management" \
demo/seed_problem_record_ticket.sh
```

Re-running with a different `SEED_CI` re-routes the same ticket; you do not need to reset between the two.

8. The demo flow

#	BEAT	WHAT TO SAY IT PROVES
1	Open the ticket; routing panel appears above the analysis	The CI resolved to an application, owning team and repo by table lookup — no model call, nothing to confirm
2	Impact analysis + effort estimate	Vague tickets get a clarifying question first, rather than a confident guess
3	Generate code	Only the files the relevance funnel selected are sent — the token panel shows scoped vs naive cost
4	Review file by file: Ask, Apply this file, Reject	Developers accept or reject one file at a time; a rejection records a reason to the ticket's audit trail and is excluded from Apply
5	Apply, then look at PolicyCore on :8501	The client's running application changed
5b	Source control panel: branch → commit → push	The change lands on a feature branch cut off <code>main</code> before anything is written, the commit is gated on the tests passing, and the push hands off to the pipeline — the flow, not a direct edit to main
6	Revert (per file or all)	Anything applied can be undone without a full demo reset
6b	Design doc: change map + Download PDF	The hand-off document carries a diagram of what the change touches, and leaves as a real PDF you can attach to the ticket
7	Draft test scenarios , edit one, approve the plan	QA reviews <i>what will be checked</i> , in prose traced to the CR's acceptance criteria, before any test code exists — and can change it

#	BEAT	WHAT TO SAY IT PROVES
8	Generate tests, run them, then the seeded-bug check	The generated tests actually catch a deliberately introduced bug
9	Run the regression suite	A human-authored suite the AI cannot write to still passes — the CR cost nothing that already worked
10	Build the traceability matrix	Every acceptance criterion, the scenarios planned for it, the tests that ran, and the result — the artifact an auditor asks for
11	Design-doc drift check	Documentation drift is detected automatically after Apply, not by remembering to press a button
12	Release notes — three audiences + the derived deployment & rollback plan	One note per reader (client / ops / user guide), and a deploy order computed from the change's own service graph
13	Download the release record, attach it to the ticket	Everything the run proved, in one PDF — including what it could <i>not</i> prove

On beats 12-13: the deployment order is **derived**, not drafted — on CR-2026-043 the plan puts policy-service before claims-service because claims-service calls it, and says why. The release record is assembled from what the run actually produced; its "Not evidenced by this release" block is the part worth pausing on, because a release document that only lists successes is marketing. **Attach to ticket** is honest about the demo default: with `JIRA_MODE=replay` there is no Jira to attach to, so the beat records the intent on the ticket timeline and says the upload was simulated. Set `JIRA_MODE=live` and it uploads for real.

On beat 5b: say plainly that the git flow is **modelled, not executed** — the panel says so on screen and the release record repeats it under "Not evidenced by this release". Nothing runs git and no remote is contacted. That is deliberate: the target apps live inside this repo and the reset scripts restore the baseline from `HEAD`, so a real commit would make them start restoring the CR instead. The point of the beat is the *shape* of the flow — branch before edit, commit gated on green tests, pipeline on push — which is what a reviewer asks about when they see an AI editing code.

Two things are worth clicking rather than describing. Press **Commit to branch** before running the tests: it refuses, and names the reason, because the gate is computed server-side from the ticket's own test results — the console cannot assert "tests passed". And open **What a real integration would have run** for the git transcript, which grows one step at a time as you take each step. If you Revert everything afterwards, the branch shows as *abandoned* rather than disappearing, and a commit already made is not unmade — in a real repo the honest undo at that point is a revert commit, not a rewritten history.

On beat 6b: the change map is **derived, not drawn by the model** — services, layers and the cross-service arrow are read from the changed-file set, so it costs no LLM call and needs no cache warming. The **NEW** badge comes from git (the file is absent from `HEAD`), which is why `ClaimRules.java` carries one on CR-2026-043 and nothing does on CR-2026-042. The PDF is rendered server-side by headless Chromium; if `playwright install chromium` has not been run on the demo machine the endpoint answers 503 and the console silently falls back to the browser's own print-to-PDF, so the button always does something.

Beats 7, 9 and 10 are the QA-facing half of the tests stage. Two things worth saying out loud when showing them:

- The regression suite (`tests/test_regression_policycore.py` , `PolicyApiRegressionTest.java`) appears in **no** target's `testgen_allowlist` . The pipeline physically cannot write to it, which is what makes "the pre-existing tests still pass" a result rather than a claim.
- In the matrix, only the scenario→test column is inferred, and it is deliberately conservative: an ambiguous pairing renders as "no automated test" rather than guessing. On CR-2026-043 two criteria legitimately land there — that is the honest answer, and a good moment to make the point that the tool reports gaps instead of hiding them.

For the full per-scenario talk track and the fallback ladder, see `demo/DEMO_TEST_GUIDE.md` .

9. Troubleshooting

SYMPTOM	CAUSE AND FIX
<code>FOREIGN KEY</code> constraint failed during a reset or seed	An old baseline whose <code>wipe_db()</code> predates the endorsements table. Fixed in the current scripts. If you hit it on an older checkout: <code>rm -f data/mockapp.db</code> , then re-run the reset.
<code>codegen</code> returned unexpected file set	A target directory under <code>apps/</code> was renamed or moved. The replay recordings contain the exact paths. Restore the directory name — see the warning in <code>apps/README.md</code> .
A beat hangs, or fails mentioning your LLM URL	A narrative beat missed its cache and called your model. Run <code>demo/warm_s3_cache.sh</code> . Confirm the endpoint with the one-liner in section 4.
Applied change crashed the portal	Expected and handled — the console shows the migration traceback with a one-click fix. You can also press Revert all .
Console UI loads but every call 401s	Not logged in, or the API on :8000 is not running. Check terminal 1.
Claims-Service errors on a claim	Policy-Service on :8081 is not up. Claims calls policy over HTTP; start #3 first.
Port already in use	<code>lsof -ti:8000 xargs kill</code> (repeat per port). Note that <code>--reload</code> is deliberately not used on the API: the reloader restarts mid-beat when codegen writes to the tree.

10. Pre-demo checklist

- ☐ `python -m pytest -q` passes
- ☐ `.env` has your provider block filled in, and the section-4 one-liner returned a reply
- ☐ All three reset scripts ran clean, in order
- ☐ `demo/warm_s3_cache.sh` ran *after* the resets
- ☐ Console reachable at :5173 ; you are logged in
- ☐ PolicyCore reachable at :8501 in a second window
- ☐ For the ClaimsPortal CR: :8081 and :8082 both responding
- ☐ You have walked beats 1–13 once, end to end, on this machine

The four applications live under `apps/`; the AI pipeline that drives them is in `s3_enhancement/`, shared clients in `common/`, and presenter scripts in `demo/`. Do not rename directories under `apps/` — see `apps/README.md`.